Ingénierie des Processus de Développement Logiciel 1

Lab React/Next 2 — Pages et navigation

Layout — Sidebar — Routing — Liens

Licence 3 GLSI — 2025–2026

Dr. El Hadji Bassirou TOURÉ
École Supérieure Polytechnique
Département Génie Informatique
Université Cheikh Anta Diop de Dakar

Informations pratiques

Durée estimée	2h (une séance)
Prérequis	Lab React/Next 1 terminé. Les 6 composants sont créés.
Rôles impliqués	Tous — chacun crée la page de sa fonctionnalité
Contexte	Repo **SénSanté** — on passe de « une page » à « une application »
Tag	`v0.1` (layout et navigation complets)
Validation	L'application a un layout avec sidebar et 6 pages navigables.

Objectifs du lab

À l'issue de ce lab, chaque étudiant sera capable de :

1. Comprendre le **layout** de Next.js (la coquille partagée par toutes les pages)
2. Créer **plusieurs pages** en utilisant le routing par dossiers
3. Construire une **sidebar** avec des liens de navigation
4. Utiliser le composant `<Link>` pour naviguer sans rechargement
5. Mettre en évidence la **page active** dans la sidebar

Table des matières

1 Introduction — De la page unique à l'application **3**

2 Le Layout : la coquille de l'application **3**
2.1 Comment fonctionne le layout ? 3

3 Le routing par dossiers **4**

4 Exercice 1 — Créer le layout avec sidebar **4**
4.1 Étape 1 : Créer le composant Sidebar 4
4.2 Étape 2 : Modifier le layout 5

4.3 Étape 3 : Vérifier 6

5 La navigation avec Link **7**

6 Exercice 2 — Chaque membre crée sa page **7**
6.1 Le Gardien : page Patients 7
6.2 Le Médecin : page Consultations 8
6.3 Le Pilote : page Dashboard 9
6.4 Le Bouclier : page Login 9
6.5 L'Oracle : page IA (placeholder) 10
6.6 Le Pilote : page Profil 10
6.7 Commit et PR 11

7 Exercice 3 — Tester la navigation **11**

8 Aide-mémoire **11**

9 Qui fait quoi ? **12**

10 Résumé et conclusion **12**

1 Introduction — De la page unique à l'application

Dans le Lab React/Next 1, vous avez assemblé tous vos composants sur **une seule page**. Le résultat est un aperçu intéressant, mais SénSanté est une **application multi-pages** : une page pour les patients, une pour les consultations, une pour le dashboard, etc.

Aujourd'hui, vous passez de « une page » à « une vraie application » avec :

- Un **layout** permanent (header + sidebar)
- **6 pages** distinctes avec du contenu propre
- Une **navigation** fluide entre les pages

2 Le Layout : la coquille de l'application

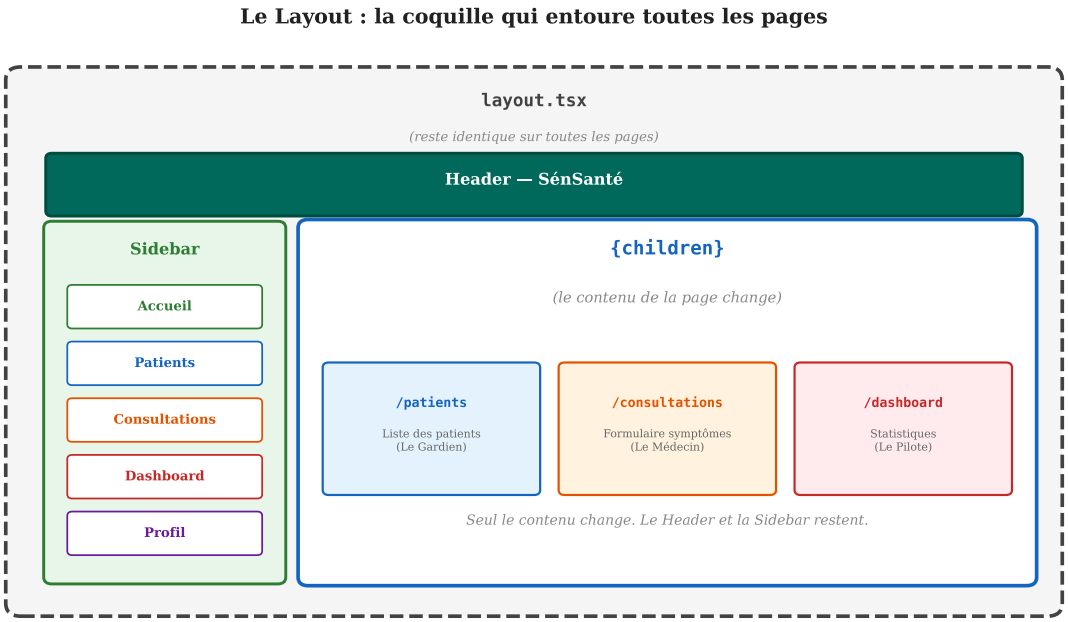

Fig. 1 : Le Layout : Header et Sidebar restent — seul le contenu (`{children}`) change.

2.1 Comment fonctionne le layout ?

Dans Next.js, le fichier `layout.tsx` définit la **structure commune** à toutes les pages. Tout ce qui est dans le layout (header, sidebar, footer) reste en place quand on navigue. Seul le contenu de la page change.

L'analogie du cadre photo

Le layout, c'est le **cadre**. Les pages, ce sont les **photos**. Quand vous changez de page, vous changez la photo dans le cadre — mais le cadre (header, sidebar) ne bouge pas.
Le contenu de la page est injecté à l'endroit du `{children}` dans le layout.

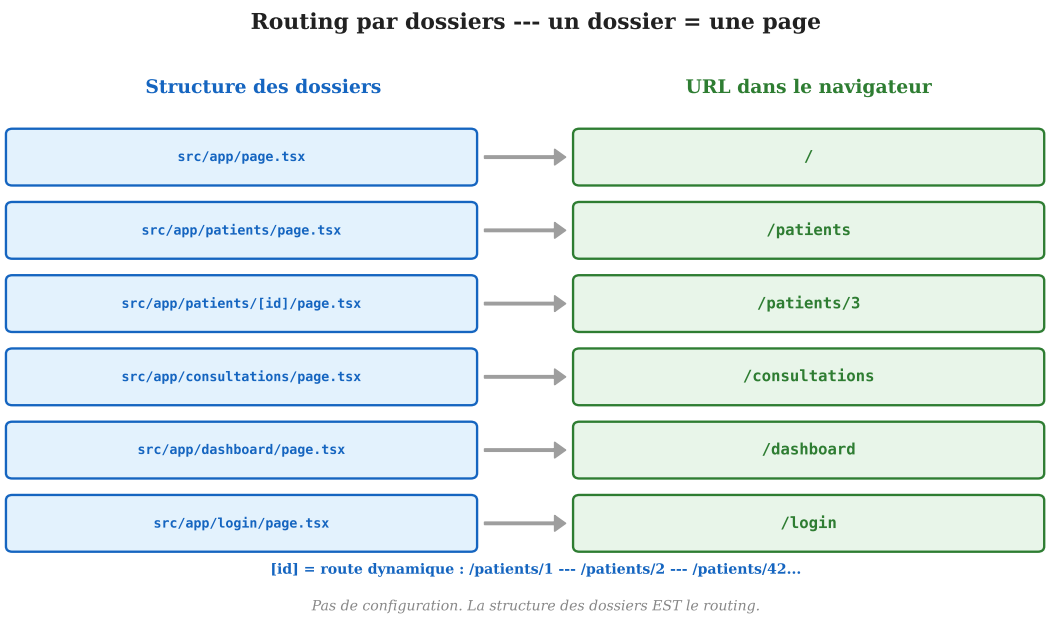

Fig. 2 : Le routing par dossiers : chaque dossier dans `src/app/` crée une route.

3 Le routing par dossiers

La règle du routing

Pour créer une page à l'URL `/patients` :

1. Créez le dossier `src/app/patients/`
2. Créez le fichier `page.tsx` dedans
3. C'est tout. L'URL `/patients` existe.

Pas de fichier de configuration. Pas de « router » à paramétrer. **La structure des dossiers est le routing.**

4 Exercice 1 — Créer le layout avec sidebar

Objectif

Transformer le layout pour ajouter un header et une sidebar permanents.

4.1 Étape 1 : Créer le composant Sidebar

L'Architecte crée le fichier `src/components/Sidebar.tsx` :

```
"use client";

import Link from "next/link";
import { usePathname } from "next/navigation";

const liens = [
  { nom: "Accueil",        href: "/",              icone: "H" },
  { nom: "Patients",       href: "/patients",      icone: "P" },
  { nom: "Consultations",  href: "/consultations", icone: "C" },
```

```
  { nom: "Dashboard",      href: "/dashboard",     icone: "D" },
  { nom: "Profil",         href: "/profil",        icone: "U" },
];

export default function Sidebar() {
  const pathname = usePathname();

  return (
    <aside className="w-64 bg-teal-800 text-white
      min-h-screen p-4">
      <nav className="space-y-2 mt-4">
        {liens.map((lien) => {
          const actif = pathname === lien.href;
          return (
            <Link
              key={lien.href}
              href={lien.href}
              className={`flex items-center gap-3 px-4
                py-3 rounded-lg transition
                ${actif
                  ? "bg-teal-600 font-bold"
                  : "hover:bg-teal-700"}`}
            >
              <span className="w-8 h-8 bg-teal-600
                rounded-full flex items-center
                justify-center text-sm">
                {lien.icone}
              </span>
              {lien.nom}
            </Link>
          );
        })}
      </nav>
    </aside>
  );
}
```

Concepts importants dans ce code

`"use client"`	Indique à Next.js que ce composant s'exécute côté navigateur (nécessaire pour `usePathname`).
`Link`	Le composant de navigation de Next.js (pas de rechargement).
`usePathname()`	Un **hook** qui retourne l'URL actuelle (`/patients` etc.).
`liens.map(...)`	Boucle sur le tableau pour générer un lien par élément.
`actif`	Vrai si l'URL actuelle correspond au lien — pour le mettre en surbrillance.

4.2 Étape 2 : Modifier le layout

Modifiez `src/app/layout.tsx` :

```
import type { Metadata } from "next";
import { Inter } from "next/font/google";
import "./globals.css";
import Header from "@/components/Header";
import Sidebar from "@/components/Sidebar";

const inter = Inter({ subsets: ["latin"] });

export const metadata: Metadata = {
  title: "SénSanté",
  description:
    "Assistant de santé communautaire avec IA",
};

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html lang="fr">
      <body className={inter.className}>
        <div className="flex flex-col min-h-screen">
          <Header />
          <div className="flex flex-1">
            <Sidebar />
            <main className="flex-1 bg-gray-50 p-8">
              {children}
            </main>
          </div>
        </div>
      </body>
    </html>
  );
}
```

Le {children} est la clé

La variable `{children}` est l'emplacement où Next.js injecte le contenu de la page. Quand vous naviguez vers `/patients`, le contenu de `src/app/patients/page.tsx` remplace `{children}`. Le Header et la Sidebar ne bougent pas.

4.3 Étape 3 : Vérifier

```
npm run dev
```

Vous devez voir le header en haut, la sidebar à gauche, et le contenu à droite. Cliquez sur les liens de la sidebar — le contenu change mais la sidebar reste.

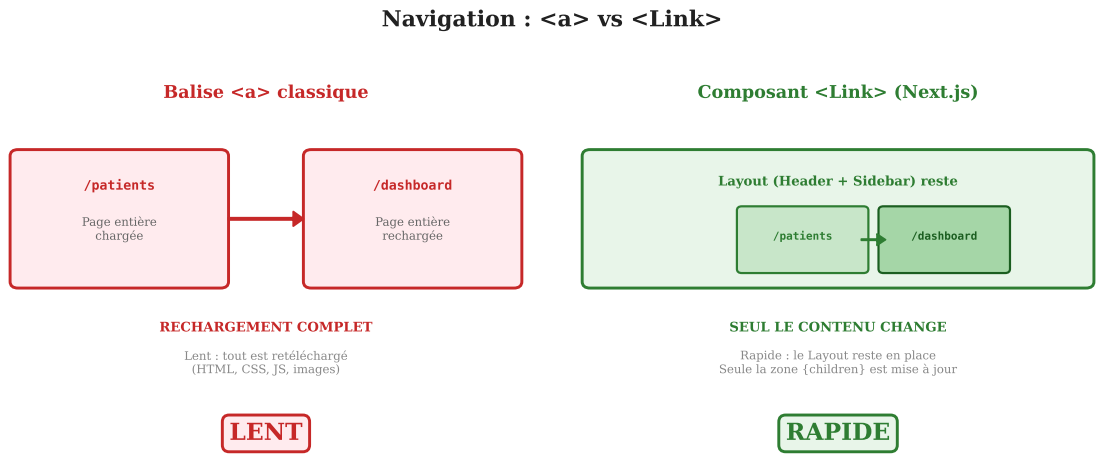

Fig. 3 : Différence entre `<a>` (rechargement complet) et `<Link>` (navigation rapide).

5 La navigation avec Link

Toujours utiliser Link — jamais <a>

Dans une application Next.js, utilisez **toujours** `<Link>` pour les liens internes :

✓ **Correct :** `<Link href="/patients">Patients</Link>`

✗ **Incorrect :** `<a href="/patients">Patients</a>`

`<Link>` fait une **navigation côté client** (rapide, pas de rechargement). `<a>` recharge toute la page (lent).

Exception : utilisez `<a>` uniquement pour les liens **externes** (vers d'autres sites).

6 Exercice 2 — Chaque membre crée sa page

Objectif

Chaque membre crée la page de sa fonctionnalité avec un contenu temporaire.

Chaque membre crée sa branche :

```
git switch main && git pull
git switch -c page/<rôle>
```

6.1 Le Gardien : page Patients

Créez `src/app/patients/page.tsx` :

```
import PatientCard from "@/components/PatientCard";

export default function PatientsPage() {
  const patients = [
    { nom: "Aminata Sow", region: "Dakar",
      age: 34, sexe: "F" as const },
    { nom: "Ibrahima Ba", region: "Thiès",
      age: 45, sexe: "M" as const },
```

```
    { nom: "Awa Diallo", region: "Saint-Louis",
      age: 28, sexe: "F" as const },
    { nom: "Cheikh Fall", region: "Ziguinchor",
      age: 52, sexe: "M" as const },
  ];

  return (
    <div>
      <h1 className="text-2xl font-bold
        text-gray-800 mb-6">
        Patients
      </h1>
      <div className="grid grid-cols-1
        md:grid-cols-2 lg:grid-cols-3 gap-4">
        {patients.map((p, i) => (
          <PatientCard key={i} {...p} />
        ))}
      </div>
    </div>
  );
}
```

Syntaxe {...p} (spread)

`{...p}` est le **spread operator**. Il « déverse » toutes les propriétés de l'objet `p` comme props du composant. C'est équivalent à écrire `nom={p.nom} region={p.region} age={p.age} sexe={p.sexe}` — en plus court.

6.2 Le Médecin : page Consultations

Créez `src/app/consultations/page.tsx` :

```
import ConsultationCard
  from "@/components/ConsultationCard";

export default function ConsultationsPage() {
  return (
    <div>
      <h1 className="text-2xl font-bold
        text-gray-800 mb-6">
        Consultations
      </h1>
      <div className="space-y-4">
        <ConsultationCard
          patient="Aminata Sow"
          date="18 mars 2025"
          symptomes="Fièvre, toux, fatigue"
          statut="termine" />
        <ConsultationCard
          patient="Ibrahima Ba"
          date="19 mars 2025"
          symptomes="Maux de tête, vertiges"
          statut="en_attente" />
      </div>
    </div>
  );
}
```

6.3 Le Pilote : page Dashboard

Créez `src/app/dashboard/page.tsx` :

```
import StatCard from "@/components/StatCard";

export default function DashboardPage() {
  return (
    <div>
      <h1 className="text-2xl font-bold
        text-gray-800 mb-6">
        Tableau de bord
      </h1>
      <div className="grid grid-cols-1
        md:grid-cols-3 gap-4">
        <StatCard titre="Patients" valeur={127}
          unite="enregistrés"
          couleur="border-teal-500" />
        <StatCard titre="Consultations" valeur={43}
          unite="ce mois"
          couleur="border-orange-500" />
        <StatCard titre="Alertes" valeur={8}
          unite="urgentes"
          couleur="border-red-500" />
      </div>
    </div>
  );
}
```

6.4 Le Bouclier : page Login

Créez `src/app/login/page.tsx` :

```
export default function LoginPage() {
  return (
    <div className="flex items-center justify-center
      min-h-[60vh]">
      <div className="bg-white p-8 rounded-lg
        shadow-md w-full max-w-md">
        <h1 className="text-2xl font-bold
          text-gray-800 mb-6 text-center">
          Connexion
        </h1>
        <div className="space-y-4">
          <input type="email" placeholder="Email"
            className="w-full p-3 border rounded-lg" />
          <input type="password"
            placeholder="Mot de passe"
            className="w-full p-3 border rounded-lg" />
          <button className="w-full bg-teal-600
            text-white py-3 rounded-lg
            hover:bg-teal-700 transition">
            Se connecter
          </button>
        </div>
```

```
        <p className="text-sm text-gray-400
          text-center mt-4">
          Fonctionnalité complète dans le Lab Auth
        </p>
      </div>
    </div>
  );
}
```

6.5 L'Oracle : page IA (placeholder)

Créez `src/app/ia/page.tsx` :

```
import AlerteIA from "@/components/AlerteIA";

export default function IAPage() {
  return (
    <div>
      <h1 className="text-2xl font-bold
        text-gray-800 mb-6">
        Assistant IA
      </h1>
      <p className="text-gray-600 mb-6">
        Saisissez les symptômes du patient pour
        obtenir un pré-diagnostic.
      </p>
      <AlerteIA
        diagnostic="Suspicion de paludisme.
          Orientation vers un centre de santé."
        confiance={78}
        niveau="urgent" />
    </div>
  );
}
```

6.6 Le Pilote : page Profil

Créez `src/app/profil/page.tsx` :

```
export default function ProfilPage() {
  return (
    <div>
      <h1 className="text-2xl font-bold
        text-gray-800 mb-6">
        Mon profil
      </h1>
      <div className="bg-white rounded-lg
        shadow-md p-6 max-w-md">
        <p className="text-gray-600">
          <strong>Nom :</strong> Moussa Diop
        </p>
        <p className="text-gray-600 mt-2">
          <strong>Rôle :</strong> Agent de santé
        </p>
        <p className="text-gray-600 mt-2">
          <strong>Région :</strong> Dakar
        </p>
```

```
      </div>
    </div>
  );
}
```

6.7 Commit et PR

Chaque membre :

```
git add src/app/
git commit -m "Page <nom-page> (<Nom de guerre>)"
git push -u origin page/<rôle>
```

L'Architecte merge les PR.

7 Exercice 3 — Tester la navigation

Objectif

Vérifier que toutes les pages fonctionnent et que la navigation est fluide.

Après merge de toutes les PR :

```
git switch main && git pull
npm run dev
```

Checklist de vérification — testez chaque lien de la sidebar :

Lien	Page attendue	Fonctionne ?
Accueil	Page d'accueil avec composants	☐
Patients	Grille de cartes patients	☐
Consultations	Liste des consultations	☐
Dashboard	3 StatCards en grille	☐
Profil	Informations du profil	☐

Vérifiez aussi que :

- Le **Header reste** en haut quand vous changez de page
- La **Sidebar reste** à gauche quand vous changez de page
- Le lien **actif** est en surbrillance dans la sidebar
- La navigation est **instantanée** (pas de rechargement)

```
git add .
git commit -m "Navigation complète - toutes les pages (v0.1)"
git tag v0.1
git push && git push origin v0.1
```

8 Aide-mémoire

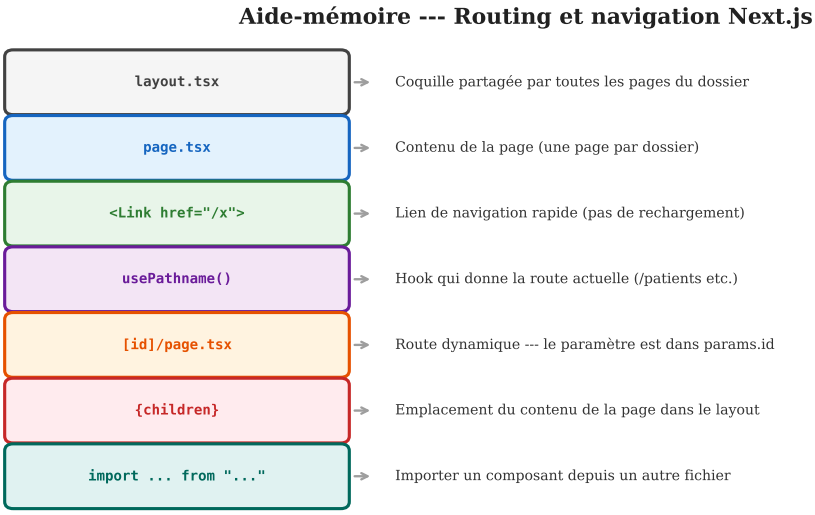

Fig. 4 : Aide-mémoire — Routing et navigation Next.js.

9 Qui fait quoi ?

Répartition des tâches — Lab React/Next 2

Nom de guerre	Tâche dans ce lab
L'Architecte	Crée le `Sidebar.tsx` et modifie le `layout.tsx`. Merge les PR. Pose le tag `v0.1`.
Le Gardien	Crée `src/app/patients/page.tsx` avec la grille de PatientCards.
Le Bouclier	Crée `src/app/login/page.tsx` avec le formulaire de connexion.
Le Médecin	Crée `src/app/consultations/page.tsx` avec les ConsultationCards.
L'Oracle	Crée `src/app/ia/page.tsx` avec le placeholder IA + AlerteIA.
Le Pilote	Crée `src/app/dashboard/page.tsx` et `src/app/profil/page.tsx`.

Workflow : `git switch -c page/<rôle>` → créer la page → `commit` → `push` → PR → merge.

Aucune séance ne se termine sans que chaque membre ait produit quelque chose.

10 Résumé et conclusion

Dans ce lab, vous avez :

1. Compris le **layout** Next.js (coquille permanente autour des pages).

2. Maîtrisé le **routing par dossiers** (un dossier = une URL).
3. Construit une **sidebar** avec navigation et lien actif en surbrillance.
4. Utilisé `<Link>` pour une **navigation instantanée** (pas de rechargement).
5. Créé **6 pages** distinctes avec du contenu propre à chaque rôle.

Prochaine étape : la base de données

L'application a maintenant une vraie structure : header, sidebar, 6 pages navigables. Mais les données sont en dur dans le code. L'étape suivante consiste à connecter les pages à la **base de données** via Prisma : créer, lire, modifier, supprimer des patients. C'est le territoire du **Gardien**.
Tag visé : `v0.2` (patients CRUD).

`v0` → `v0.1` → `v0.2` → `v0.3` → …→ `v1.0`

setup → **layout** → patients → auth → consultations → IA → dashboard → docker → livraison

Rapport individuel

Rapport individuel — obligatoire

Chaque membre rédige un **rapport individuel** de **1 à 2 pages** :

1. **Ce que j'ai fait :** page créée, composants utilisés, navigation testée.
2. **Les difficultés rencontrées :** routing, layout, usePathname, Link.
3. **Comment je les ai résolues :** documentation Next.js, aide d'un camarade.
4. **Ce que j'ai appris :** layout, routing par dossiers, navigation SPA, sidebar.

Nom du fichier : `rapport-lab-react2-PRENOM-NOM.pdf`
Preuve : capture d'écran montrant la sidebar avec le lien actif et le contenu de votre page.

SénSanté — Tag v0.1 ✓

Layout, sidebar, 6 pages. L'application prend forme.

Prochaine étape : les données (patients CRUD).

Chaque commit compte. Chaque membre compte.